Task-15:- Technical Documentation(Nginx – Enabling HTTPS & Securing with Basic SSL/TLS Configuration)

1. Introduction

1.1 Purpose

The purpose of this document is to detail the technical implementation of a secure web server environment using Nginx on AWS. It serves as a comprehensive guide for **CTOs, Developers, Solution Architects, and DevOps Engineers** to understand the deployment, security hardening, and SSL configuration of a Linux-based web host.

1.2 Scope

This documentation covers the end-to-end process of provisioning an EC2 instance, installing the Nginx web server, generating a self-signed SSL certificate using OpenSSL, configuring firewall rules via UFW, and verifying HTTPS connectivity.

---

2. Overview of DevOps Project

2.1 Definition

This project implements the "Security" and "Reliability" facets of DevOps. By automating SSL generation and server configuration, we ensure that data in transit is encrypted, reducing the risk of man-in-the-middle attacks.

2.2 Key Principles (AWS Well-Architected Framework)

We adhered to the **6 Pillars** as follows:

- **Security:** Achieved by implementing SSL/TLS encryption and strict UFW firewall rules.
- **Reliability:** Nginx is configured as a systemd service to ensure high availability and auto-restart capabilities.
- **Performance Efficiency:** Used a `t3.micro` instance, optimized for burstable workloads.
- **Cost Optimization:** Leveraged Free Tier-eligible resources.
- **Operational Excellence:** Utilized Nginx configuration testing (`nginx -t`) before reloading services.
- **Sustainability:** Minimized resource footprint by using a lightweight Ubuntu Server image.

2.3 Goals and Objectives

- Deploy a functional Nginx web server.
- Enable HTTPS via a 2048-bit RSA self-signed certificate.

- Implement security headers (X-Frame-Options, X-Content-Type-Options).

2.4 Constraints and Assumptions

- **Technology:** Restricted to Ubuntu 22.04/24.04 and Nginx.
- **Assumptions:** User has SSH access to the AWS instance and sudo privileges.

---

3. Architecture and Infrastructure

3.1 Infrastructure Overview

The infrastructure consists of an **Amazon EC2 Instance** running in a Public Subnet within a VPC, protected by an AWS Security Group and an OS-level firewall (UFW).

3.2 Environments

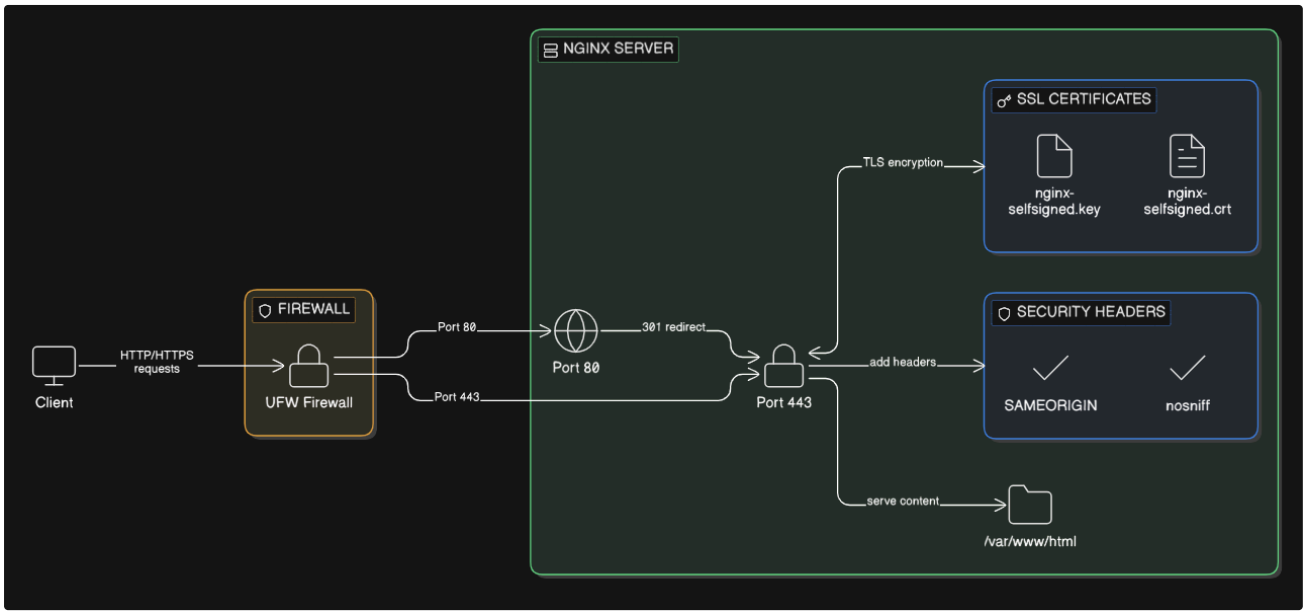
- **Production:** The live web server accessible via the Public IP: `35.175.201.236`.

3.3 Application Architecture & User Request Flow

User Flow: User (Browser) → HTTPS (Port 443) → UFW Firewall → Nginx (SSL Termination) → Web Content (`/var/www/html`).

3.4 Tools and Technologies

- **Cloud:** AWS (EC2, VPC).
- **Web Server:** Nginx.
- **Security:** OpenSSL (X.509), UFW (Uncomplicated Firewall).
- **OS:** Ubuntu Linux.



---

4. Development Practices

4.1 Version Control

Configuration files (like `default` Nginx config) are managed via local backups and can be integrated into Git repositories for infrastructure tracking.

4.2 Continuous Integration

Configuration was validated using the `sudo nginx -t` command to ensure syntax accuracy before deployment.

```
ubuntu@ip-172-31-14-251:~$ ls /etc/nginx/ssl
nginx-selfsigned.crt  nginx-selfsigned.key
ubuntu@ip-172-31-14-251:~$ sudo chmod 600 /etc/nginx/ssl/nginx-selfsigned.key
ubuntu@ip-172-31-14-251:~$ ls -l /etc/nginx/ssl
total 8
-rw-r--r-- 1 root root 1363 Mar 15 10:10 nginx-selfsigned.crt
-rw------- 1 root root 1704 Mar 15 10:09 nginx-selfsigned.key
ubuntu@ip-172-31-14-251:~$ sudo nano /etc/nginx/sites-available/default
ubuntu@ip-172-31-14-251:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-172-31-14-251:~$ sudo systemctl reload nginx
ubuntu@ip-172-31-14-251:~$ sudo ufw status
Status: inactive
ubuntu@ip-172-31-14-251:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
ubuntu@ip-172-31-14-251:~$ sudo ufw allow 'Nginx Full'
Rule added
Rule added (v6)
ubuntu@ip-172-31-14-251:~$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
Nginx Full                 ALLOW       Anywhere
Nginx Full (v6)            ALLOW       Anywhere (v6)

ubuntu@ip-172-31-14-251:~$
```

---

5. Deployment and Release Management

5.1 Deployment Strategy

A **Rolling Update** approach was used: Nginx was installed and the service status was verified as "Active" before applying SSL configurations.

```
ubuntu@ip-172-31-14-251:~$ sudo apt install nginx -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nginx is already the newest version (1.24.0-2ubuntu7.6).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ip-172-31-14-251:~$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-03-15 08:50:08 UTC; 1h 14min ago
       Docs: man:nginx(8)
    Process: 14069 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 14071 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 14101 (nginx)
      Tasks: 3 (limit: 1008)
     Memory: 2.4M (peak: 5.3M)
        CPU: 28ms
     CGroup: /system.slice/nginx.service
             ├─14101 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─14103 "nginx: worker process"
             └─14104 "nginx: worker process"

Mar 15 08:50:08 ip-172-31-14-251 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Mar 15 08:50:08 ip-172-31-14-251 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-14-251:~$
```

5.2 Configuration Management

The SSL certificate was generated using OpenSSL with the following parameters:

- **Algorithm:** RSA 2048
- **Validity:** 365 Days
- **Location:** Surat, Gujarat, India.

```
ubuntu@ip-172-31-14-251:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/nginx/ssl/nginx-selfsigned.key -out
/etc/nginx/ssl/nginx-selfsigned.crt
...+...+..........++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*...+...+..+.+..........+...+...+.........+++++
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*........+.+.....+......+.+.....+...+.+......+............+.............
.+..+.........+........+.....................+.+.............+.+.....+....+..+....+...+...+.............+...............+...+..+.
...+..........+...+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
...+...+.+...+..+...+......+......+...+....+..........+....+...+........++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
+++++++++++++.+.....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++.+.........+......+...+.+......+...+..+...+.+..
+....+......+............+.....................+.....+...+..+.......+.....+.........+.......+...+..+....+.....+..........+...+..
.+...............+..............+...+......+...........+.............+..+.......+...+..+.+.........+..+....+.........+...........+.+
......+..+.......+.............+......+.+.....+.......+......+.....+.......+...+..........+.............+.....+...+.........+.+....
...............+....+...+..+..........+...+..+.+...+..+.........+...+...+.......+.....+.+..+....+........+......+.......+........+..
.......+.+....................+.+.........+.....+.......+.........+...+......+..+...............+....+.....+.+...+.....+......+...+..
.+...+....+...........+.....+......+...+.......+...+..+.............+.....................+...+...+...+.....+.+.........+..+....+...
..+...+...+...+....+................+...+.........+..+...+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:IN
State or Province Name (full name) [Some-State]:Gujarat
Locality Name (eg, city) []:Surat
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Demo
Organizational Unit Name (eg, section) []:35.175.201.236
Common Name (e.g. server FQDN or YOUR name) []:35.175.201.236
Email Address []:
ubuntu@ip-172-31-14-251:~$
```

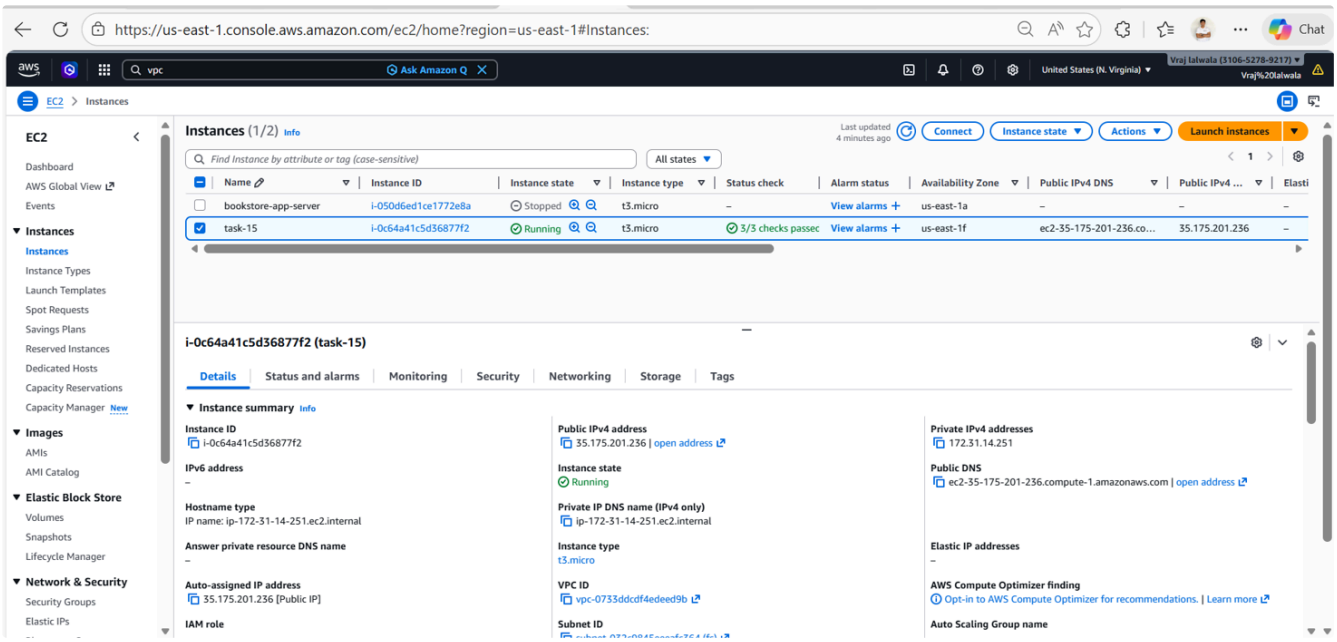
---

6. Monitoring and Incident Management

6.1 Monitoring Tools and Metrics

Standard monitoring includes `systemctl status nginx` to track uptime and `ls -l /etc/nginx/ssl` to verify the presence of security credentials.

6.2 Alerting and Notification



---

7. Security & Compliance

- **SSL Configuration:** Nginx was configured to listen on Port 443 with paths pointing to `nginx-selfsigned.crt`.
- **Security Headers:** Added `X-Frame-Options` to prevent clickjacking.
- **Firewall:** UFW was enabled to allow "Nginx Full" (Ports 80 & 443).

---

8. Cost Optimization

- **Inputs:** Utilized `t3.micro` instance type.
- **TCO Analysis:** Total monthly cost is $0.00 under the AWS Free Tier (up to 750 hours).
- **Right-Sizing:** The instance uses only 2.4MB of memory for the Nginx process, perfectly sized for a static web host.

---

10. Implementation Timeline

10.1 Milestones and Deliverables

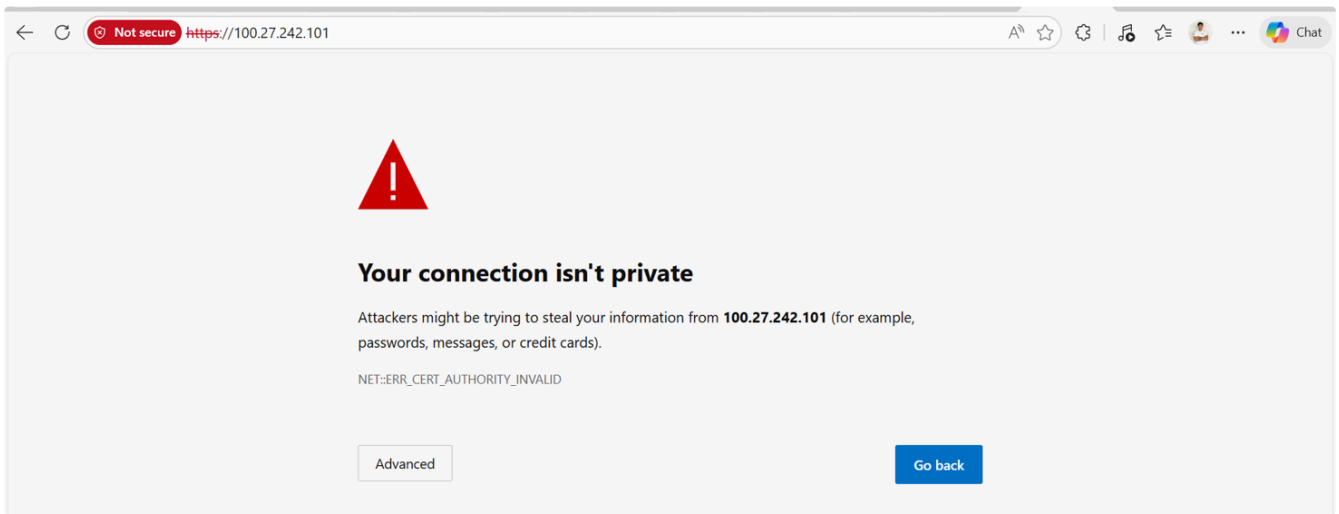
- **Phase 1:** EC2 Provisioning & Nginx Installation (Duration: 30 mins)
- **Phase 2:** SSL Generation & Configuration (Duration: 45 mins)
- **Phase 3:** Security Hardening (UFW & Headers) (Duration: 30 mins)
- **Final Delivery:** March 15, 2026.

---

11. Continuous Improvement

11.1 Feedback and Retrospectives

The current "Not Secure" browser warning is due to the self-signed nature of the certificate.



11.2 Iterative Enhancements

The next iteration will involve implementing **Certbot (Let's Encrypt)** for a CA-signed certificate once a domain name is pointed to the IP.

---

12. Conclusion

12.1 Summary

The project successfully deployed a hardened Nginx server on AWS. We verified that despite the self-signed warning, the encryption is active and the server is responsive.

12.2 Next Steps

- Map a Route 53 domain to the Public IP.
- Automate the deployment using an Ansible playbook.

---

13. Customer Acknowledgement

Name: Pushkar Khatavkar

Title: DevOps Engineer

Date: March 15, 2026